CS 3113 – Introduction to Operating Systems
Fall 2026 – Project 1

Multilevel Feedback Queue Simulator

Due Thursday, October 22, 2026 at 11:59 PM

AI Restriction. Do not upload this handout to an AI tool. The allowed uses are listed under AI Policy, just before the design report.

Overview

In Project 1 you will build a CPU scheduling simulator in C++ that models a three-level **multilevel feedback queue** (MLFQ). Processes start in the highest queue and are demoted when they consume the allotment of that queue. Higher queues always have priority over lower queues and **preempt** them. Inside Q0, different priorities are preemptive; processes that share a priority share the CPU by Round-Robin with quantum q_0.

This is the first project in which you must design a **Process Control Block** (PCB). The kernel of a real OS stores everything it knows about a process in a PCB. Your simulator is a tiny kernel: if a fact about a process has to survive a context switch, it belongs in the PCB.

DO NOT UPLOAD TO AI

Assumptions:

- Every process is CPU-bound (no I/O waits, no blocking).
- There is a single CPU core.
- One burst (burst) unit is one time unit on the CPU.
- Context-switch cost is zero.
- Processes may arrive at different times, including after the CPU has already started running.

Learning Goals

- Design a PCB as a C++ `struct` or class and justify every field.
- Represent three ready queues whose disciplines are different: preemptive priority with RR among equals, SRTF, and FCFS.
- Implement preemption between queues and inside Q0 / Q1.
- Compute response time, waiting time, turnaround time, and CPU utilization from the trace.
- Produce output that matches a required format exactly (this is how Gradescope will grade you).

Queue Rules

Queue	Algorithm	Allotment	What happens
Q0	Preemptive priority; RR among equals	q_0 (input)	Every process *enters Q0* at its arrival time. Smaller priority number = higher urgency. Different priorities: a running Q0 process is preempted as soon as a *strictly* higher-priority process is in Q0. Same priority: those processes share the CPU by **Round-Robin** with time quantum q_0. A same-priority arrival does not preempt the runner; it joins the RR tail for that priority. When a process uses a full q_0 quantum in Q0 without finishing, it is demoted to Q1 (it does not get another Q0 turn).
Q1	Shortest Remaining Time First (SRTF)	q_1 (input)	Among processes in Q1, run the one with the smallest remaining burst. Ties: earlier time of entry into Q1, then smaller PID. After a process has consumed q_1 CPU units *while in Q1* without finishing, it is demoted to Q2.
Q2	First Come First Served (FCFS)	none	Run the process that entered Q2 earliest until it finishes. Record the time a process enters Q2; that is its FCFS key. A Q2 process may be preempted by Q0 or Q1, but it keeps its place at the head of Q2.

DO NOT UPLOAD TO AI

Preemption between levels. Q0 is strictly above Q1, and Q1 is strictly above Q2. If the CPU is running a Q1 or Q2 process and any process is present in a higher queue, the running process is preempted immediately (at the current time boundary) and stays in its queue.

Priority applies only in Q0. Q1 does not look at priority numbers. Q2 does not look at priority numbers. A low-priority process that has already dropped to Q2 can be overtaken by a later high-priority arrival, because that arrival enters Q0.

Round-Robin among equal priorities in Q0. Think of Q0 as one RR ready list per priority value. Always run a process from the highest-priority nonempty list. Inside that list, the discipline is ordinary RR: the process at the head runs, a new same-priority arrival is appended, and a process that is preempted by a *higher* priority keeps its leftover quantum and stays in that list. Burning the quantum still means *demote to Q1*, not "go to the back of Q0 for another turn." So two priority-3 jobs each get one q_0 slice in RR order, then drop; they do not rotate forever at Q0.

Leftover quantum. Each process gets a q_0-unit budget when it enters Q0. Preemption by a higher-priority process does *not* reset that budget. When the interrupted process is selected again, it resumes the leftover slice. Demotion happens when the budget hits zero, even if that budget was spent across several dispatches. The Q1 allotment q_1 bursts (bursts) the same way.

The PCB you must design

A defensible PCB for this simulator needs, at minimum, a place to store:

- identity: PID, original burst, arrival time, priority;
- progress: remaining burst, which queue the process is in now, leftover Q0 budget, CPU consumed in Q1, time it entered Q1, time it entered Q2;
- metrics: time of first dispatch (for response time), finish time (for turnaround and waiting);
- state: at least NEW / READY / RUNNING / TERMINATED.

You choose the C++ types. A `struct PCB` with those fields is enough. You may use a class. Document the fields in the design report — that page is graded.

For the three ready collections you also choose the structures: a queue / deque is the natural fit for Q2; Q1 must support "find the shortest remaining"; Q0 must support "highest priority first, and Round-Robin among that priority" (one FIFO per priority level is enough). Arrays of size ≤ 10 are acceptable because $N \leq 10$. So are `std::deque`, `std::vector`, or a tiny linked list of PCB pointers. Do not use a library scheduler.

DO NOT UPLOAD TO AI

Simulator loop (required event order)

Implement a time-driven loop. At each integer time t, do the steps in this order. Hidden tests assume this order when two things happen at the same instant.

1. **Admit.** Every process whose arrival time equals t enters Q0, in the order the processes appeared in the input. Set its Q0 budget to q_0.
2. **Preempt if needed.** If a process is running and it is no longer the one that *should* run (see step 3), preempt it. It stays in its current queue. Close the current output slice with `preempted`.
3. **Select.**
   - If Q0 is nonempty: the waiting Q0 process with the *smallest priority number* (highest urgency). If several share that priority, take them Round-Robin: the one that has been waiting at the head of that priority's RR list. A same-priority arrival is appended and does not kick the runner off mid-quantum.
   - Else if Q1 is nonempty: smallest remaining burst; then earliest Q1-entry time; then smallest PID.
   - Else if Q2 is nonempty: the head of Q2.
   - Else the CPU is idle.
4. **Run** one time unit, or jump the clock to the next arrival if the CPU is idle.
5. **Finish or demote** at the end of the unit, in this order: (a) remaining burst is $0 \rightarrow$ terminate; (b) else if the process is in Q0 and its Q0 budget is $0 \rightarrow$ demote to Q1; (c) else if the process is in Q1 and CPU consumed in Q1 equals $q_1 \rightarrow$ demote to Q2.

The first time a process is selected in step 3 is the moment used for response time.

Metrics

Let A_p be the arrival time of process p, W_p its original burst, S_p the time of its first dispatch, and F_p the time it terminates.

$$\begin{aligned} \text{Response time}(p) &= S_p - A_p \\ \text{Turnaround time}(p) &= F_p - A_p \\ \text{Waiting time}(p) &= \text{Turnaround time}(p) - W_p \\ \text{CPU utilization} &= \frac{\text{busy time units}}{T} \end{aligned}$$

T is the time at which the last process terminates (the makespan). Idle gaps count in the denominator and not in the numerator. Print utilization as the unreduced fraction `busy/T` — write `21/25`, not `0.84` and not `1/1` when the values happen to be equal (`94/94` is the required form).

Print per-process metrics in **increasing PID order**, not in finish order.

DO NOT UPLOAD TO AI

Your tasks

AI Policy (required)

- You **may not** upload this handout, or any substantial copy of the project description, to an AI model or coding assistant.
- You **may not** ask an AI tool to write your simulator, your PCB, or your queue code. A generated scheduler that you submit as your own is not your work.
- You **may** ask an AI tool to help you read the input from the text file (the redirected `std::cin` format: q_0, q_1, N, then N lines of PID, work, arrival, and priority). The scheduling decisions must still be yours.
- You **may** ask an AI tool to help troubleshoot code you already wrote. Show it the failing input and your own attempt (for example: "this loop skips every other process — why?").
- You **may** ask an AI tool to clarify a sample after you have traced it by hand. Do not ask it to produce the trace for you.
- If you use an AI tool, add an appendix that lists the prompts, what did not work and why, and the hardest bug you fixed yourself. The appendix does not count against the three-page limit.

1. Design report (one to three pages, plus an AI appendix if needed)

Write a report of one to three pages that covers:

- The PCB: every field, its C++ type, and why it exists.
- The data structure used for Q0, for Q1, and for Q2.
- Two edge cases you actually ran. An edge case is a legal input that sits on a boundary the obvious path does not exercise. It is not a malformed file. It is a situation your loop can reach: two events at the same time, an empty queue, a quantum expiring on the same tick a process finishes, or a tie that must be broken. A run in which one process arrives at time 0 and runs alone does not count.

Do not reuse Sample 1 or Sample 2. A label such as "empty queue" is not enough. For each edge case give:

- the input, small enough to trace by hand;
- why it is an edge case, not an ordinary run;
- what your simulator does at the critical time: which queue wins, and whether the slice ends in `terminated`, `preempted`, `to Q1`, or `to Q2`;
- a Gantt chart for that input;
- response time, waiting time, turnaround time, and CPU utilization. Waiting time is time spent in Q0, Q1, or Q2 while the process is not running.

These two inputs are the `inputA.txt` and `inputB.txt` you submit on Canvas, with the outputs your program produces.

If you used any generative AI (ChatGPT, Claude, Gemini, Copilot, Grok, ...), add an appendix of one or more extra pages that lists the prompts, what did not work and why, the most challenging bug you hit, and how you fixed it. The appendix does not count against the three-page limit.

2. Implementation

Write the simulator in C++ and name the file `project1.cpp`. Read from standard input, write to standard output. Comment the PCB, the three queues, and the context-switch / demotion steps so a reader can follow the design without guessing.

The program must compile with

```
g++ -std=c++11 project1.cpp -o project1
```

and run with

```
./project1 < input.txt
```

DO NOT UPLOAD TO AI

3. Testing

Run the sample inputs before you submit. Gradescope will also use hidden inputs. Matching the published pairs is necessary and not sufficient.

Build two additional inputs of your own (`inputA.txt`, `inputB.txt`) and the outputs your program produces for them. Those four files go to Canvas with the report.

Input format

Standard input, no extra blank lines:

- Line 1: two positive integers q_0 and q_1 — the Q0 quantum and the Q1 allotment.
- Line 2: a positive integer N, the number of processes ($1 \le N \le 10$).
- Next N lines: four positive integers (arrival may be 0) separated by a single space:
  - **PID** — unique among the N processes.
  - **burst** — total CPU burst, 1–80.

– **Arrival** — time the process enters Q0, an integer ≥ 0.

– **Priority** — integer ≥ 1. **1 is highest urgency.** Larger numbers are lower urgency.

Hidden tests obey the same constraints. PIDs need not be $1..N$ in order. You may assume the input is well formed.

Output format

One slice per contiguous run of the same process in the same queue, or one idle gap. Use these four endings and nothing else:

```
Time a-b: P<id>, from Q0 to Q1
Time a-b: P<id>, from Q1 to Q2
Time a-b: P<id>, from Qk, terminated
Time a-b: P<id>, from Qk, preempted
Time a-b: Idle
```

Then a blank line, then the four metric blocks exactly as shown in the samples (headings, `P<id> = <int>`, utilization fraction). No extra commentary. No trailing spaces. One newline at the end of the file.

DO NOT UPLOAD TO AI

Sample 1 — priority preemption in Q0

Input:

```
10 5
2
1 20 0 2
2 5 3 1
```

P1 arrives at time 0 with priority 2 and starts in Q0. At time 3, P2 arrives with priority 1 (higher urgency) and preempts P1 — this is a different-priority preemption, not RR. P1 keeps a leftover Q0 budget of 7. P2 finishes inside Q0. P1 then burns the rest of its Q0 budget, drops to Q1, burns the Q1 allotment, drops to Q2, and finishes. (Two processes of priority 2 would instead have taken turns of q_0 by Round-Robin and each demoted after its slice.)

Interval	Who	Queue	What ends the slice
0–3	P1	Q0	preempted by P2 (priority $1 < 2$)
3–8	P2	Q0	terminated (burst $5 < q_0$)
8–15	P1	Q0	leftover budget used; demoted
15–20	P1	Q1	$q_1 = 5$ consumed; demoted
20–25	P1	Q2	terminated

Output:

```
Time 0-3: P1, from Q0, preempted
Time 3-8: P2, from Q0, terminated
Time 8-15: P1, from Q0 to Q1
Time 15-20: P1, from Q1 to Q2
Time 20-25: P1, from Q2, terminated

Response Time
P1 = 0
P2 = 0
```

```
Turnaround Time
P1 = 25
P2 = 5

Waiting Time
P1 = 5
P2 = 0

CPU Utilization Rate
25/25
```

Sample 2 — idle gap and a walk through all three queues

Input:

```
5 5
2
1 6 0 1
2 15 10 1
```

P1 uses a full Q0 quantum, then finishes in Q1 (one unit of burst left, well under the Q1 allotment). The CPU sits idle from 6 to 10. P2 arrives at 10 and walks Q0 → Q1 → Q2.

Output:

```
Time 0-5: P1, from Q0 to Q1
Time 5-6: P1, from Q1, terminated
Time 6-10: Idle
Time 10-15: P2, from Q0 to Q1
Time 15-20: P2, from Q1 to Q2
Time 20-25: P2, from Q2, terminated

Response Time
P1 = 0
P2 = 0

Turnaround Time
P1 = 6
P2 = 15

Waiting Time
P1 = 0
P2 = 0

CPU Utilization Rate
21/25
```

DO NOT UPLOAD TO AI

Submission

On Canvas (upload as separate files, do not zip):

- `DesignReportProject1.pdf`
- `inputA.txt`, `inputB.txt`
- `outputA.txt`, `outputB.txt`

On Gradescope:

- `project1.cpp`

Compile notes

Linux `g++ -std=c++11 project1.cpp -o project1`
`./project1 < input.txt`

macOS `clang++ -std=c++11 project1.cpp -o project1`
If `clang++` is missing: `xcode-select --install`

Windows Use the VS Code or Windows Powershell terminal with the same `g++` command. Setup notes: `https://code.visualstudio.com/docs/cpp/config-mingw`

To capture an output file:

```
./project1 < input1.txt > my1.txt
diff -u output1.txt my1.txt
```

Rules

This is an individual assignment. The syllabus late policy applies: up to five late days, five percent per day, then no credit. The program must compile with GCC/G++ and `-std=c++11`. Several unpublished inputs will be used in grading. Write the scheduler yourself. High-level questions to a classmate or to an AI tool are allowed; a pasted implementation is not.

CS 3113 Fall 2026 Project 1. Three-level MLFQ with a student-designed PCB. Q0 is preemptive priority with RR among equals; Q1 is SRTF with an allotment; Q2 is FCFS.

DO NOT UPLOAD TO AI